

ソフトウェア開発と AI システム工学におけるタスク分割の重要性

Arthur Delfino Duarte
電気工学学士号（電子工学および通信工学専攻）
arthur.d.duarte@live.com

抄録—タスク分割はソフトウェア工学における基本原理であり、最近では大規模言語モデル（LLM）に基づく人工知能システムにも応用されている。本稿では、人間の脳の認知の限界、コンテキストスイッチングのコスト、および専門化が生産性と品質をどのように向上させるかについて考察する。従来のソフトウェアチームとマルチエージェント AI アーキテクチャの間の類似性を示し、複雑な問題をより小さな専門化されたタスクに分割することで、より信頼性が高く、保守が容易で、スケーラブルなシステムが実現されることを実証する。

キーワード—マルチエージェントシステム、LLM、ソフトウェア工学、タスク分割、認知負荷、コンテキストスイッチング

I. はじめに

世界中のほぼすべてのテクノロジー企業が、一人ですべてを行うことを期待するのではなく、専門化されたチームで作業する理由がある。一見すると、一人の開発者がシステムのすべての部分を把握しているため、プロジェクトをより適切に管理できるように思えるかもしれない。しかし、複雑さが増すにつれて、この戦略はますます非効率になる。

興味深いことに、言語モデル（LLM）を使用する際にもまったく同じ現象が発生する。多くのユーザーはすべてを行うための単一の巨大なプロンプトを構築しようとするが、実際には、最新の AI システムは専門化されたエージェント間で責任を分割する方がはるかに効率的である。原理はまったく同じである。複雑な問題をより小さな問題に分割することで、より高速で、より信頼性が高く、保守が容易なシステムが実現される。

本稿は、ソフトウェア工学と人工知能の間のこの交差点を探求し、認知負荷、専門化、およびマルチエージェントアーキテクチャに関する理論的基礎を、実践的な応用例とともに提示する。

II. タスク分割の基礎

A. 人間の脳の生物学的限界

最初の重要な概念は、誰もプロジェクト全体を作業記憶に保持できないことを理解することである。開発中には、アーキテクチャ、データベース、API、フロントエンド、認証、セキュリティ、UX、ドキュメント、テスト、デプロイ、監視、最適化など、数十もの同時活動が存在する。これらの各領域は異なる種類の推論を必要とする。

この困難はスキルの不足ではなく、認知の限界に起因する。人間の作業記憶は同時に限られた量の情報しか維持できない。ソフトウェア工学の生産性に関する研究によると、開発者は、重大な中断なしに長時間作業できる日を生産的な日と見なしている。379 人のプロの開発者を対象とした研究では、中断と頻繁なコンテキストスイッチが、生産性に有害と認識される主要な要因の一つとして挙げられた。

複雑さは線形に増大するのではなく、コンポーネント間の相互作用とともに増大する。小規模なシステムでは、開発者は実質的にすべてのコンテキストを頭の中に保持できる。新しい機能が出現するにつれて、コードの量だけでなく、モジュール間の関係、依存関係、およびアーキテクチャ上の決定の数も増加する。

B. 三つの基本的役割

一人でシステムを開発する場合でも、通常は三つの異なる役割を担う。第一は計画者であり、何を構築するか、どのように構築するか、どのアーキテクチャを使用するか、どの技術を採用するかを決定する責任を持つ。その焦点は戦略的であり、コードは書かず、考える。第二は実行者であり、計画をコードに変換す

る。その目標は生成することであり、実装中にアーキテクチャを再設計すべきではない。第三は監督者であり、継続的に問いかける：機能しているか？バグはないか？コードはクリーンか？安全か？文書化されているか？要件は満たされているか？監督者はレビューアとして機能する。

一人ですべてを行うと、これらの三つの精神モード間を継続的に切り替えることになる。これにより認知疲労が生じ、生成される作業の品質が低下する。

C. コンテキストスイッチングの見えるコスト

開発者が認証機能を実装しているとして。このタスク中、彼らは頭の中でログインフロー、トークン、データベース、暗号化、セッション、API、クライアント要件を保持している。突然、別のモジュールにバグが発生する。今、彼らはこのすべてのコンテキストをアンロードし、まったく異なるものをロードする必要がある。バグを修正した後、以前の精神状態を再構築する必要がある。

コンテキストスイッチごとに、脳は以前の状態を精神的に再構築する必要がある。これには時間がかかる——非常に多くの時間がかかる。ソフトウェア工学の研究によると、中断後にコンテキストを完全に回復するには数分かかる可能性があり、頻繁な中断は生産性を著しく低下させる。人間とコンピュータの相互作用の研究者は、この現象を数十年にわたって研究してきた。結果によると、頻繁な中断はストレスを増大させ、精神的負荷を高め、元のタスクの再開を困難にし、生産性の認識を低下させる可能性がある。これらの影響は、プログラミング、アーキテクチャ、コードレビューなどの創造的および分析的な活動に特に関連する。

D. 認知負荷の概念

すべての脳には限界がある。バックパックを想像するとわかりやすい：各タスクが重さを追加する。アーキテクチャ、データベース、フロントエンド、Docker、セキュリティ、ドキュメント、テスト——これらの各領域が総精神負荷

に貢献する。バックパックが重くなると、忘れっぽさ、バグ、誤った決定、疲労が生じる。

タスク分割はこの重さを分散する。一人ですべてを担う代わりに、各チームメンバーは自分の部分だけを担うことで、中断の減少、手戻りの削減、エラーの低減、および速度の向上が実現される。これは単により多くの人がいるからではなく、専門化が存在するからである。

III. ソフトウェアチームにおける作業分割

A. チームが問題を解決する方法

各人が主要な責任を持つチームを想像してみよう：アーキテクト、バックエンド、フロントエンド、QA、DevOps、ドキュメント。各専門家が自分の焦点を維持し、全員がすべてをしようとするよりも効率的な結果が得られる。作業分割とは、単に人を分割するのではなく、責任を分割することである。

全員がすべてを行う 10 人のチームは、作業の重複、アーキテクチャの競合、手戻り、および一貫性のない決定を生み出す傾向がある。一方、各役割が明確に定義されたチームは——同じ人数であっても——曖昧さを減らし、調整を改善するため、はるかに効率的になる傾向がある。

B. 生産パイプライン

大企業は工場のように機能する。要件はアーキテクチャを通過し、それが開発に供給され、その後にテスト、デプロイ、監視が続く。各段階には明確な基準があり、エラーが後続のフェーズに進むのを防ぐ。

エラーのコストは、検出が遅れるほど指数関数的に増加する。計画中に発見されたバグのコストは 1 倍である。開発中は 3 倍、テスト中は 10 倍、本番環境に到達した後は 100 倍である。エラーが早期に検出されるほど、修正コストは低くなる——これが、適切に構造化されたプロセスに常時レビューが存在する理由である。

C. 並列処理

一人で作業する場合、タスクを順次実行する必要がある：データベース、次にAPI、次にフロントエンド、次にテスト。総時間は100%である。一方、チームは並行して作業できる：データベース、API、フロントエンド、テストを同時に実行できる。これは、作業分割によってもたらされる最大の生産性向上の一つである。

D. 専門化

航空宇宙産業では、一人のエンジニアがエンジン、機体、電子機器、組み込みソフトウェア、油圧システムをすべて単独で設計することを期待する者はいない。各専門家は特定の領域で知識を深める。ソフトウェア工学も同じ方向に進化してきた。専門化は個人の認知的努力を減らし、各領域内での決定の質を向上させる。

1975年、Frederick P. Brooks Jr. はソフトウェア工学で最も影響力のある書籍の一つとされる『The Mythical Man-Month』を出版した。彼の最もよく知られた原則であるブルックスの法則は次のように述べている：遅れているプロジェクトに人を追加すると、さらに遅れる傾向がある。これは、新しいメンバーがアーキテクチャを学び、ビジネスルールを理解し、内部パターンを把握し、他のメンバーとコミュニケーションを取る必要があるためである。Brooks は、コミュニケーションチャネルの数がおおよそ $n(n-1)/2$ の式で増加することを示し、調整にもコストがかかることを明らかにした。これは大規模なチームが悪いということではなく、組織化、明確な役割、および適切に定義されたアーキテクチャが必要であることを意味する。

5,414 件の初期研究を分析し、102 件の関連論文を選択した系統的レビューによると、締切圧力の下では、多くのチームがより迅速に生産するが、ソフトウェアの品質は低下する傾向がある。この結果は、生産性は書かれたコードの量だけでなく、システムの品質と持続可能性によっても測定されるべきであることを強調している。

IV. 人工知能への原理の適用

A. 巨大プロンプト問題

これまで議論してきたすべては、言語モデルにもほぼ完璧に適用される。多くのユーザーは、単一のモデルに要件分析、アーキテクチャ設計、コード作成、テスト実行、ドキュメント作成、セキュリティレビュー、パフォーマンス最適化をさせようとして、巨大なプロンプトを構築する。実際には、これは一人の専門家に大規模チームのすべての役割を同時に実行させることと同等である。

数千行のプロンプトを想像してみよう。モデルは、目的、ルール、コンテキスト、制約、標準、例、アーキテクチャを同時に維持する必要がある。これにより、要件の忘れ、指示の矛盾、責任の混在、および一貫性のないコード生成の可能性が高まる。大きなコンテキストウィンドウがあっても、多くの責任がモデルの注意を競い合うと、推論の品質が低下する可能性がある。

巨大なプロンプトの問題

非常に大きなプロンプトは、モデルに一度に多くの情報を与えずぎてしまいます。



- 要件を忘れる
- 指示が矛盾する
- 一貫性のない回答
- エラーやハルシネーションの増加



小さな部分に分けることで、品質と信頼性が向上します。

図1. 巨大プロンプト問題：単一のプロンプトがすべての責任を網羅しようとする。

B. エージェント工学

すべてに責任を持つ単一のエージェントではなく、それぞれが非常に明確な機能を持つ複数の専門化されたエージェントを作成する。典型的な例には、スーパーバイザー、プランナー、バックエンド実行者、フロントエンド実行者、データベース実行者、テスター、ドキュメンターが含まれる。各エージェントはその機能に必要なコンテキストのみを受け取り、注意散漫を減らし、応答品質を向上させる。

どのエージェントもすべてをしようとはしないことに注目してほしい。各エージェントは自身の特定の問題のみを解決する。これは人間のチームにおける作業分割の原理とまったく同じである。

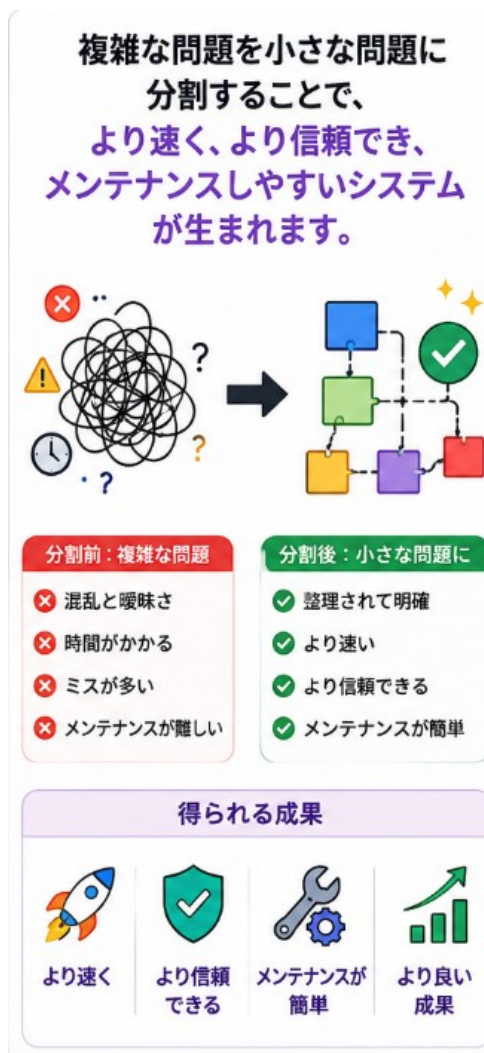


図2. 複雑な問題をより小さな専門化された問題に分割。

C. エージェント間のフロー

マルチエージェントシステムの典型的なフローは次のように機能する：ユーザーがスーパーバイザーにリクエストを行い、スーパーバイザーはそれをプランナーに転送する。プランナーは問題をタスクに分割し、専門化された実行者（バックエンド、フロントエンド、データベース）に分配する。その後、レビューアが一貫性をチェックし、テスターが品質を検証し、ドキュメンターが文書を作成する。最後に、統合された応答がユーザーに届けられる。

フローの各ステップは明確に定義されており、どのエージェントもシステムのすべての領域を深く知る必要はない。各エージェントはより小さく、より焦点を絞ったコンテキストで作業する。

D. サブエージェントの利点

マルチエージェントアプローチにはいくつかの利点がある。第一に、専門化：各エージェントは必要なコンテキストのみを受け取り、注意散漫を減らし、品質を向上させる。第二に、コンテキストの削減：100 ページのプロンプトの代わりに、各エージェントは約 10 ページを受け取り、品質を向上させ、幻覚を減らす。第三に、再利用性：テストエージェントはどのプロジェクトでも使用でき、ドキュメンテーションエージェントも同様である——再利用可能なコンポーネントが作成される。

第四に、保守の容易さ：ドキュメントが変更されても、ドキュメンターのみが置き換えられ、残りは同じままである。第五に、そして最も重要なことに、並列処理：複数のエージェントが同時に作業できる——バックエンド、フロントエンド、データベース、ドキュメントを同時に処理でき、総時間を大幅に削減する。

E. テックリードとしてのオーケストレーター

人間のチームでは、作業を調整する責任者がいることが多い：テックリード、アーキテクト、またはエンジニアリングマネージャーである。AI システムでは、

この役割はオーケストレーターエージェントによって実行される。オーケストレーターはユーザーのリクエストを解釈し、問題をサブタスクに分割し、各サブタスクを適切な専門家に転送し、結果を統合し、納品前に一貫性を検証する。オーケストレーターは最高のプログラマーやライターである必要はなく、その主な機能はワークフローを調整することである。

V. 実践例とアナロジー

A. 従来手法 vs. 現代的手法

従来の手法では、ユーザーは LLM に直接リクエストを送信し、LLM は単一の応答でシステム全体を生成しようとする。これは一人の開発者がすべてを単独で行おうとすることに相当し——結果はしばしば一貫性がなく、表面的で、エラーが発生しやすくなる。

現代的な手法では、オーケストレーターがリクエストを受け取り、タスクリストを生成するプランナーに転送する。これらのタスクは専門化された実行者（API、データベース、フロントエンド）に分配される。その後、QA が結果を検証し、ドキュメンターが文書を作成し、最終応答がユーザーに届けられる。はるかに組織的で信頼性が高い。

B. 建設工事とのアナロジー

建物を建設することを想像してみよう。設計図を描き、構造を計算し、電気システムを設置し、配管を行い、壁を建設し、塗装し、品質をレビューし、報告書を発行するために、たった一人を雇うだろうか？おそらく違う。専門化されたプロフェッショナルが存在する：アーキテクト、構造エンジニア、電気技師、配管工、左官、塗装工、検査官。各専門家は自分の領域を習得し、他の人々と協調して作業する。

マルチエージェント AI システムも同じ論理に従う：オーケストレーターエージェントが専門家に作業を分配し、専門家が結果を統合する。複雑なプロジェクトの理想的なアーキテクチャは、すべてを行うスーパーエージェントではなく、協調して作業する専門化されたエージェントのチームである——ちょうどよく組

織化されたソフトウェアエンジニアリングチームのように。

C. ログ分析システムの例

Ollama、OpenCode、およびローカルモデルを使用する環境では、分類器とプランナーを調整するオーケストレーターでマルチエージェントアーキテクチャを構成できる。分類器はログリーダーに供給し、プランナーは Linux 専門家を調整する。結果はエラー検出とイベント関連に流れ、レポート生成器に供給され、技術レビューアを通過して、最終応答が生成される。

各エージェントは、短く、客観的で、高度に専門化されたプロンプトを使用でき、コンテキスト負荷を減らし、保守を容易にする。このパラダイムは、LLM ベースのシステムが規模と複雑さにおいて進化するにつれて、ますます重要になる傾向がある。

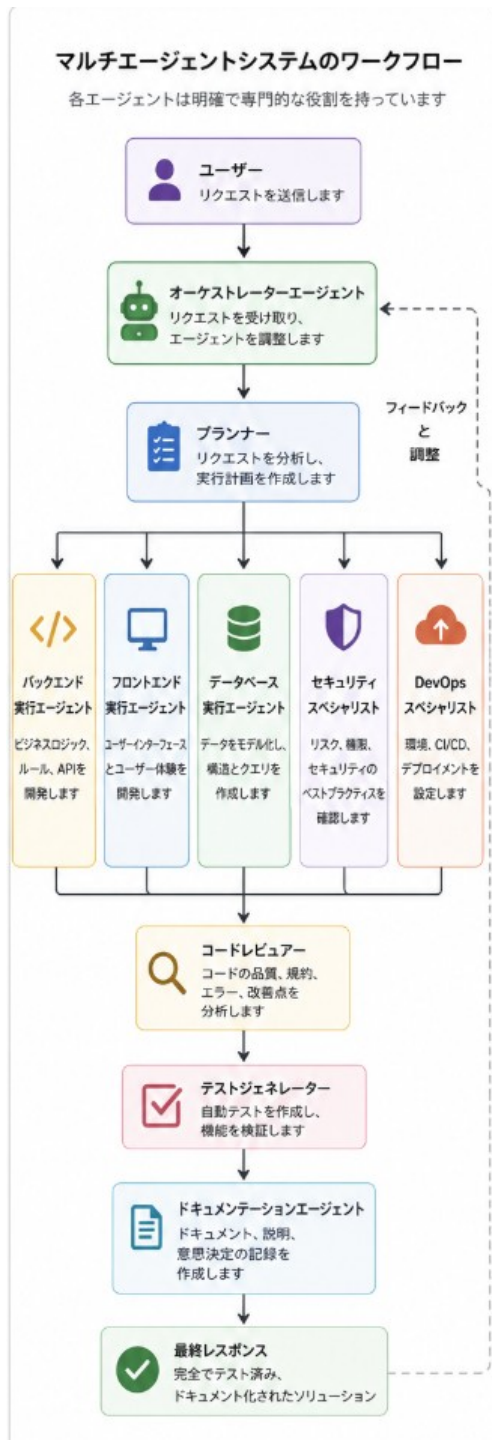


図3. オーケストレーターと専門家によるマルチエージェントシステムのワークフロー。

VI. 結論

ソフトウェア工学、認知心理学、およびコンピュータ科学からの証拠は、同じ方向を示している：作業が明確な責任に分解されると、複雑なシステムはより効

率的になる。これは人間にも、そしてますます人工知能にも当てはまる。

すべての責任を一人の開発者や単一のプロンプトに集中させる代わりに、最新のアーキテクチャは、オーケストレーターによって調整される専門家間で機能を分散する。このアプローチは認知負荷を減らし、決定の質を向上させ、エラーの伝播を減らし、テストと保守を容易にし、継続的なシステム進化を可能にする。

LLMのコンテキストでは、このパラダイムは特に強力である。専門化されたエージェントのエコシステムは、単一のスーパープロンプトよりも、より予測可能で、監査可能で、スケーラブルな方法で複雑な問題を解決できる。この変化は、ソフトウェア工学で発生した移行と同様である：孤独なプログラマーの姿から、機能、プロセス、および専門化に基づくコラボレーションによって組織されたチームへの移行である。

ソフトウェア工学の進化は、専門化、責任の分割、および適切に定義されたプロセスが、すべての機能を一人に集中させるよりも高品質なシステムを生み出すことを示してきた。現代のAI工学はまったく同じ原理に従っており、LLMベースのシステムが規模と複雑さにおいて進化し続けるにつれて、この傾向は確固たるものになるだろう。

参考文献

- F. P. Brooks Jr. The Mythical Man-Month: Essays on Software Engineering. Addison-Wesley, 1975.
- WikiBooks. LaTeX/Mathematics. URL: en.wikibooks.org/wiki/LaTeX/Mathematics (2025年5月15日アクセス) .
- P. Lima, E. Guerra, P. Meirelles, L. Kanashiro, H. Silva, and F. Silveira. "A Metrics-Driven Approach for Quality Improvement in Software Development." Journal of Systems and Software, 2017.
- J. Schell. The Art of Game Design: A Book of Lenses. 2nd ed. A. K. Peters, Ltd., 2014.
- P. Lima, E. Guerra, M. Nardes, A. Mocci, G. Bavota, and M. Lanza. "An Annotation-based Approach for Detecting Software Defects." In: Proceedings of the International Conference on Software Engineering, 2017.

S. C. Müller and T. Fritz. "Stuck and Frustrated or In Flow and Happy: Sensing Developers' Emotions and Progress." In: Proceedings of the IEEE/ACM 37th IEEE International Conference on Software Engineering, 2015.

M. Czerwinski, E. Horvitz, and S. Wilhite. "A Diary Study of Task Switching and Interruptions." In: Proceedings of the SIGCHI Conference on Human Factors in Computing Systems, 2004.

行言語モデル（LLM）、インフラストラクチャ自動化、および最新アプリケーションのためのスケーラブルなアーキテクチャに集中しています。

著者

Arthur Delfino Duarte は、Instituto Nacional de Telecomunicações (Inatel) で電気工学の学士号を取得し、IT インフラストラクチャ、コンピュータネットワーク、データセンター、通信において豊富な経験を持ち、Huawei、Nokia、Net/Claro、Surf Telecom などの企業で勤務してきました。

キャリアを通じて、Linux および Windows システム、ルーター、スイッチ、データセンターインフラストラクチャ、監視、災害復旧、大規模通信サービスを含む、クリティカルな環境の導入および運用プロジェクトに参加してきました。現在は、ソフトウェア工学、バックエンド工学、DevOps、クラウドコンピューティング、分散システム、ソフトウェアアーキテクチャの知識を深め、Java、Spring Boot、Docker、Kubernetes、REST API、リレーショナルデータベース、ローカル実行人工知能を用いた実践的なプロジェクトを開発しています。

また、ネットワーキング、Linux、クラウドコンピューティング、プロジェクト管理における認定資格とトレーニングを保有しており、Red Hat、Microsoft、Google、AWS からのトレーニング、および Nokia、Juniper、Microsoft Azure テクノロジーのプロフェッショナル認定資格を含んでいます。

主な関心は、ソフトウェア工学、高可用性インフラストラクチャ、システム信頼性、可観測性、ソフトウェア開発に応用された人工知能、ローカル実